

KULLIYAH OF ENGINEERING

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 2, 2025 / 2026

WEEK 6 : SMART SURVEILLANCE CAMERA USING ESP32-CAM

SECTION 2

GROUP 4

INSTRUCTORS : ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO

Date of Submission: 15th April 2026

NO	NAME	MATRIC NUMBER
1	Hannah Irdeena binti Azrulhafiz	2410018
2	Azzam Azhari Noureldein Elamin	1827295
3	Kamarul Zafri bin Kamarul Ariffin	2323189

ABSTRACT

This module focuses on the integration of IoT and actuator systems by using the ESP-32 CAM module. The primary objective was to successfully video stream over a Wi-Fi network and implement a horizontal camera via a servo motor to simulate automated motion tracking. This involves configuring the ESP32-CAM with a wireless data transmission and using the Pulse Width Modulation (PWM) to precisely control the actuator.

The results demonstrate successful low-latency video streaming and responsive mechanical control of the camera's view. This confirms the versatility of the ESP32-CAM for various mechatronic applications, especially between microcontroller, sensor and wireless communication. In a nutshell, the project provides a comprehensive and practical foundation for designing scalable IoT-based solutions.

TABLE OF CONTENTS

NO.	TOPIC	PAGE
1.	ABSTRACT	2
2.	INTRODUCTION	4
TASK 1		
3.	MATERIALS AND EQUIPMENT	5
4.	EXPERIMENTAL SETUP	5
5.	METHODOLOGY	6
6.	DATA COLLECTION	12
7.	DATA ANALYSIS	12
8.	RESULTS	12
9.	DISCUSSION	12
10.	RECOMMENDATIONS	13
11.	CONCLUSIONS	14
TASK 2		
12.	MATERIALS AND EQUIPMENT	14
13.	EXPERIMENTAL SETUP	15
14.	METHODOLOGY	16
15.	DATA COLLECTION	21
16.	DATA ANALYSIS	22
17.	RESULTS	22
18.	DISCUSSION	22
19.	RECOMMENDATIONS	23
20.	CONCLUSIONS	24
21.	REFERENCES	25
22.	APPENDICES	25
23.	ACKNOWLEDGEMENT	26
24.	CERTIFICATE OF AUTHENTICITY	26

Introduction

The experiment's purpose is to design and implement a smart surveillance system using the ESP32-CAM module, demonstrating the integration of IoT wireless video streaming with mechatronic servo motor control across two levels of automation: manual pushbutton controlled panning and automated face tracking panning.

The ESP32-CAM is a compact microcontroller module that combines an ESP32-S3 chip with an OV2640 camera sensor, built-in Wi-Fi and Bluetooth capabilities, and a microSD card slot. The module is capable of capturing video frames and streaming them over a local Wi-Fi network using an HTTP web server, representing a core principle of IoT systems enabling remote monitoring and control through network connectivity. The OV2640 camera sensor supports a built-in face detection algorithm implemented in hardware, which can identify human faces within the captured frame and return their bounding box coordinates (x, y, width, height) without requiring external processing.

To enable horizontal panning, a servo motor was integrated and controlled using Pulse Width Modulation (PWM). The ESP32's LEDC (LED Control) peripheral generates the 50Hz PWM signal required by standard hobby servos, where the pulse width determines the angular position: 500µs corresponds to 0° and 2400µs corresponds to 180°.

In Part A, a pushbutton serves as a manual trigger so that the servo only pans when the button is physically pressed, providing user-directed control over the camera's viewing angle. In Part B, the system is upgraded to autonomous tracking by reading the X-coordinate of the detected face from the camera's face detection data and using it to calculate the required servo adjustment, creating a closed-loop tracking system that continuously centres the camera on the subject's face.

It is expected that in Part A, the ESP32-CAM will stream live video over Wi-Fi and the pushbutton will reliably trigger servo panning. In Part B, it is expected that the servo will automatically follow a detected face by adjusting its angle based on the face's horizontal position within the frame, maintaining the face near the centre of the video feed with minimal oscillation.

TASK 1

1.1 MATERIALS AND EQUIPMENT

1. Arduino Uno
2. ESP32-CAM Module
3. USB to micro USB adapter
4. Servo motor
5. Pushbuttons
6. Jumper Wire
7. Breadboard

1.2 EXPERIMENTAL SETUP

1. Launch the **Arduino IDE** on your computer to begin the setup.
2. Navigate to the board manager and select the **AI Thinker ESP32-CAM** profile.
3. Choose the appropriate **COM port** that matches the USB programmer's connection.
4. Flash the **servo control and pushbutton script** onto the ESP32-CAM module.
5. Use the **ESP32-CAM MB USB Programmer** to bridge the connection between the module and the laptop for data transfer.
6. Supply power to the **servo motor** using the 5V output pin from the Arduino.
7. Wire the servo motor by connecting the signal line to **GPIO 12**, power to **5V**, and the ground to **GND**.
8. Install a **pushbutton** between a designated pin (such as **GPIO 2**) and GND, ensuring the code enables an internal pull-up resistor to register inputs.
9. After booting, the module will join the local **Wi-Fi network**, allowing you to view the live video stream through a web browser using the device's **IP address**.
10. Activate the **pushbutton** to rotate the servo, which pans the camera and confirms that the manual override system is functioning correctly.

1.3 METHODOLOGY

1.3.1 Circuit Assembly

a. ESP32-CAM to FTDI (Base Setup)

ESP32-CAM Pin	FTDI Pin
U0T	RX
U0R	TX
5V	5V
GND	GND
IO0	GND (connected during upload only)

b. Pushbutton Connections

Pushbutton Pin	Connected To
One leg	GPIO (P13)
Other leg	GND

c. Servo Motor

Servo Wire	Connected To
Orange	GPIO12 (ESP32-CAM)
Red	5V (Arduino Uno)
Brown	GND

1.3.2 Programming Logic

- a. Prepare hardware and circuit.
- b. The code was modified in Arduino IDE to include pinMode(13, INPUT_PULLUP) and an if statement for the servo movement.
- c. GPIO 0 was connected to the GND to enter Flash Mode.
- d. Upon reset (by disconnecting the IO0), the module connects to the Wi-Fi network and displays its IP address.
- e. The web interface is opened in a browser to view the live stream and the pushbutton was pressed to confirm that the camera only moved when it was pressed.

1.3.3 Code Used

```
#include "esp_camera.h"

#include <WiFi.h>

#include <ESP32Servo.h>

#define CAMERA_MODEL_AI_THINKER

#include "camera_pins.h"

const char* ssid = "*****";

const char* password = "*****";

// Servo configuration

static const int servoPin = 12;

Servo myServo;

int servoAngle = 90;

int servoDirection = 1; // 1 = increasing, -1 = decreasing
```

```

// Pushbutton configuration

static const int buttonPin = 13;


void startCameraServer();


void setup() {

    Serial.begin(115200);

    Serial.setDebugOutput(true);


    // Camera configuration (same as CameraWebServer example)

    camera_config_t config;

    config.ledc_channel = LEDC_CHANNEL_0;

    config.ledc_timer = LEDC_TIMER_0;

    config.pin_d0 = Y2_GPIO_NUM;

    config.pin_d1 = Y3_GPIO_NUM;

    config.pin_d2 = Y4_GPIO_NUM;

    config.pin_d3 = Y5_GPIO_NUM;

    config.pin_d4 = Y6_GPIO_NUM;

    config.pin_d5 = Y7_GPIO_NUM;

    config.pin_d6 = Y8_GPIO_NUM;

    config.pin_d7 = Y9_GPIO_NUM;

    config.pin_xclk = XCLK_GPIO_NUM;

    config.pin_pclk = PCLK_GPIO_NUM;

    config.pin_vsync = VSYNC_GPIO_NUM;

    config.pin_href = HREF_GPIO_NUM;

```



```

config.pin_sccb_sda = SIOD_GPIO_NUM;

config.pin_sccb_scl = SIOC_GPIO_NUM;

config.pin_pwdn = PWDN_GPIO_NUM;

config.pin_reset = RESET_GPIO_NUM;

config.xclk_freq_hz = 20000000;

config.frame_size = FRAMESIZE_UXGA;

config.pixel_format = PIXFORMAT_JPEG;

config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;

config.fb_location = CAMERA_FB_IN_PSRAM;

config.jpeg_quality = 12;

config.fb_count = 1;


if(psramFound()){

    config.jpeg_quality = 10;

    config.fb_count = 2;

    config.grab_mode = CAMERA_GRAB_LATEST;

}


esp_err_t err = esp_camera_init(&config);

if (err != ESP_OK) {

    Serial.printf("Camera init failed with error 0x%x", err);

    return;

}


sensor_t * s = esp_camera_sensor_get();

```

```

if(s->id.PID == OV3660_PID){

    s->set_vflip(s, 1);

    s->set_brightness(s, 1);

    s->set_saturation(s, -2);

}

if(config.pixel_format == PIXFORMAT_JPEG){

    s->set_framesize(s, FRAMESIZE_QVGA);

}


// Servo setup

ESP32PWM::allocateTimer(0);

myServo.setPeriodHertz(50);

myServo.attach(servoPin, 500, 2400);

myServo.write(servoAngle);


// Pushbutton setup

pinMode(buttonPin, INPUT_PULLUP);


// Wi-Fi connection

WiFi.begin(ssid, password);

WiFi.setSleep(false);

while (WiFi.status() != WL_CONNECTED) {

    delay(500);

    Serial.print(".");

}

```

```

Serial.println("");

Serial.println("WiFi connected");


startCameraServer();


Serial.print("Camera Ready! Use 'http://");
Serial.print(WiFi.localIP());
Serial.println("' to connect");
}


void loop() {
  // Servo panning only when button is pressed
  if (digitalRead(buttonPin) == LOW) {
    servoAngle += servoDirection * 2;
    if (servoAngle >= 180) {
      servoAngle = 180;
      servoDirection = -1;
    } else if (servoAngle <= 0) {
      servoAngle = 0;
      servoDirection = 1;
    }
    myServo.write(servoAngle);
    delay(30);
  }
  delay(10);
}

```

}

1.4 DATA COLLECTION

Button State	Servo Behavior	Angle Range
Not pressed	Stationary	-
Pressed and held	Moves left to right to left	$0^{\circ} \rightarrow 180^{\circ} \rightarrow 0^{\circ}$
Released mid-pan	Stays at current angle	Fixed

1.5 DATA ANALYSIS

The pushbutton data confirms the manual control logic functioned as intended. With a 2° step every 30ms, the calculated panning speed is approximately $67^{\circ}/s$. The system responded instantaneously to button presses and holds, providing reliable user-directed surveillance scanning without interfering with the 15, 20 fps video stream.

1.6 RESULTS

- The ESP32-CAM successfully streamed live QVGA video over Wi-Fi accessible via a web browser.
- The pushbutton-controlled servo panning operated exactly as designed: stationary when released, smooth sweeping at $\sim 67^{\circ}/s$ when held.
- Video streaming was uninterrupted during servo operation.

1.7 DISCUSSION

We successfully updated the system so that the camera only pans when a user presses a button, rather than moving constantly on its own. To make this work reliably, we used an internal pull-up resistor, which keeps the electrical signals steady and prevents the motor from moving accidentally. Even while streaming live video, the ESP32-CAM was able to process these button presses and control the motor at the same time, proving it can handle several tasks at once without crashing.

During testing, we did notice a few small issues, such as the motor shaking slightly or moving more than once after a single press. These glitches were likely caused by "button bounce," which happens when the mechanical parts of a switch create extra electrical noise during a press. Despite these minor hiccups, the motor generally followed the instructions in our code, and the system behaved the way we planned.

We also found that the way we provided power affected how smoothly the motor moved. Since the motor was drawing power from the main board, the voltage would drop slightly whenever it started moving, causing some stuttering. In the future, we could fix this by giving the motor its own power source or adding a filter to clean up the signal. Overall, this project was a success, as it showed how to combine user input with mechanical movement in a compact electronic setup.

1.8 RECOMMENDATIONS

To improve the reliability and performance of the system in future iterations, the following recommendations are proposed based on the challenges encountered during the experiment:

1. Implementing Independent Power Management

It is highly recommended to use a dedicated external 5V power supply for the servos rather than relying on the Arduino's 5V pin. During testing, powering the servos directly from the microcontroller caused significant jittering and triggered system resets. These issues stem from the servos' high inrush and stall current, which often exceeds the capacity of a

standard USB port. Utilizing an isolated power source ensures consistent voltage, stabilizes the system, and prevents the hardware from failing under load.

2. Optimizing Control Logic and Feedback Loops

To achieve stable tracking, the control software should be carefully calibrated to ensure a proper "negative feedback" loop. Initial testing showed that the servos reacted incorrectly, moving away from targets rather than toward them due to an accidental positive feedback loop. Future setups should prioritize the systematic analysis of serial data to identify these logic errors early. Inverting the `map()` function within the code is an effective way to correct the direction of motion, ensuring the system responds accurately to sensor inputs.

1.9 CONCLUSION

The completion of Task 1 demonstrated the integration of a manual trigger into an automated system. Interfacing a pushbutton with GPIO 13 effectively transitions from continuous panning to a random manual-based motion control. The task further demonstrates the precision of the Pulse Width Modulation (PWM) with the servo motor's horizontal range between its minimum and maximum limit.

TASK 2

2.1 MATERIALS AND EQUIPMENT

1. Arduino Uno
2. ESP32-CAM Module
3. USB to micro USB adapter
4. Servo motor
5. Jumper Wire
6. Breadboard

2.2 EXPERIMENTAL SETUP

1. Launch the **Arduino IDE** on the laptop to prepare the development environment.
2. Navigate to the board manager and designate the **AI Thinker ESP32-CAM** as the target hardware.
3. Identify and select the specific **COM port** associated with the USB programmer.
4. Flash the **face detection and servo control firmware** onto the ESP32-CAM module.
5. Establish a connection between the **Arduino** and the laptop to provide a common power source and interface.
6. Plug the **ESP32-CAM MB USB programmer** into the laptop to facilitate communication with the module.
7. Link the servo's signal wire (orange) to **GPIO 12** on the ESP32-CAM to enable control signals.
8. Route the servo's power (red) and ground (brown) wires to the **5V and GND pins** on the Arduino, respectively.
9. Configure the ESP32-CAM to join the **local Wi-Fi network** to allow for wireless data transmission.
10. Monitor the **Serial Monitor** output to identify the assigned IP address required for web access.
11. Observe the system as the servo automatically executes **horizontal panning** once a face is identified within the camera's field of view.
12. Enter the ESP32-CAM's **IP address** into a web browser to open the system's control interface.
13. Monitor the live video stream to confirm that the **servo tracks movement** accurately whenever faces are detected in the frame.

2.3 METHODOLOGY

2.3.1 Circuit Assembly

a. ESP32-CAM to FTDI (Base Setup)

ESP32-CAM Pin	FTDI Pin
U0T	RX
U0R	TX
5V	5V
GND	GND
IO0	GND (connected during upload only)

b. Pushbutton Connections

Pushbutton Pin	Connected To
One leg	GPIO (P13)
Other leg	GND

c. Servo Motor

Servo Wire	Connected To
Orange	GPIO12 (ESP32-CAM)
Red	5V (Arduino Uno)
Brown	GND

2.3.2 Programming Logic

- a. Initialize and prepare setup, camera sensor and servo PWM at 50Hz for automated tracking.
- b. The code was modified in Arduino IDE to include esp_camera.h for face detection/recognition and variables (e.g face_detected boolean) are declared.
- c. Run the code and continuously fetch frames from the camera
- d. Face Tracking:
 - i. If Face=True: Stop panning and move servo to detected faces
 - ii. If Face=False: Continue default bidirectional movement.
- e. Regulate the loop to ensure the video stream remains stable when detection is active.

2.3.3 Code Used :

```
#include "esp_camera.h"

#include <WiFi.h>

#include <ESP32Servo.h>

#include <dl_lib.h>

#define CAMERA_MODEL_AI_THINKER

#include "camera_pins.h"

const char* ssid = "*****";

const char* password = "*****";
```

```

static const int servoPin = 12;

Servo myServo;

int servoAngle = 90;


const float Kp = 0.3;

const int frameCentreX = 120;

const int deadZone = 40;

const int maxStep = 5;


bool faceDetected = false;

int noFaceCount = 0;


void startCameraServer();


void setup() {

    Serial.begin(115200);

    Serial.setDebugOutput(true);


    camera_config_t config;

    config.ledc_channel = LEDC_CHANNEL_0;

    config.ledc_timer = LEDC_TIMER_0;

    config.pin_d0 = Y2_GPIO_NUM; config.pin_d1 = Y3_GPIO_NUM;

    config.pin_d2 = Y4_GPIO_NUM; config.pin_d3 = Y5_GPIO_NUM;

    config.pin_d4 = Y6_GPIO_NUM; config.pin_d5 = Y7_GPIO_NUM;

    config.pin_d6 = Y8_GPIO_NUM; config.pin_d7 = Y9_GPIO_NUM;

```

```

config.pin_xclk = XCLK_GPIO_NUM; config.pin_pclk = PCLK_GPIO_NUM;

config.pin_vsync = VSYNC_GPIO_NUM; config.pin_href = HREF_GPIO_NUM;

config.pin_sccb_sda = SIOD_GPIO_NUM; config.pin_sccb_scl = SIOC_GPIO_NUM;

config.pin_pwdn = PWDN_GPIO_NUM; config.pin_reset = RESET_GPIO_NUM;

config.xclk_freq_hz = 200000000;

config.frame_size = FRAMESIZE_240X240;

config.pixel_format = PIXFORMAT_RGB565; // Required for face detection

config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;

config.fb_location = CAMERA_FB_IN_PSRAM;

config.jpeg_quality = 12;

config.fb_count = 2;


esp_err_t err = esp_camera_init(&config);

if (err != ESP_OK) { return; }


sensor_t * s = esp_camera_sensor_get();

s->set_framesize(s, FRAMESIZE_240X240);

s->set_face_detect(s, 1); // Enable face detection


ESP32PWM::allocateTimer(0);

myServo.setPeriodHertz(50);

myServo.attach(servoPin, 500, 2400);

myServo.write(servoAngle);


WiFi.begin(ssid, password);

```

```

WiFi.setSleep(false);

while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }

Serial.println("\nWiFi connected");

startCameraServer();

}

void loop() {

    camera_fb_t * fb = esp_camera_fb_get();

    if (!fb) { delay(100); return; }

    dl_matrix3du_t *image_matrix = dl_matrix3du_alloc(1, fb->width, fb->height, 3);

    if (!image_matrix) { esp_camera_fb_return(fb); return; }

    fmt2rgb888(fb->buf, fb->len, fb->format, image_matrix->item);

    box_array_t *boxes = face_detect(image_matrix, NULL);

    if (boxes && boxes->len > 0) {

        faceDetected = true;

        noFaceCount = 0;

        box_t *face = &(boxes->box[0]); // Get largest/closest face

        int faceCentreX = face->box_p[0] + (int)(face->box_p[2] / 2);

        int error = faceCentreX - frameCentreX;

        if (abs(error) > deadZone) {

            float deltaAngle = Kp * error;

```

```

    if (deltaAngle > maxStep) deltaAngle = maxStep;

    if (deltaAngle < -maxStep) deltaAngle = -maxStep;

    servoAngle += (int)deltaAngle;

    if (servoAngle < 0) servoAngle = 0;

    if (servoAngle > 180) servoAngle = 180;

    myServo.write(servoAngle);

}

Serial.printf("Face: cx=%d, err=%d, servo=%d\n", faceCentreX, error, servoAngle);

} else {

    noFaceCount++;

    if (noFaceCount > 30) faceDetected = false;

}

if (boxes) dl_lib_free(boxes->box);

if (boxes) dl_lib_free(boxes);

dl_matrix3du_free(image_matrix);

esp_camera_fb_return(fb);

delay(100);

}

```

2.4 DATA COLLECTION

2.5 DATA ANALYSIS

The tracking speed analysis revealed a system limitation. The face detection processing requires ~85ms, plus a 100ms programmed delay, resulting in a ~200ms correction cycle. During fast walking, the subject's face moved faster than the camera could correct, causing the face to leave the frame entirely.

2.6 RESULTS

- The face detection algorithm successfully returned real-time bounding box coordinates.
- The proportional tracking algorithm automatically followed a subject's face during normal walking speeds (1.0m distance).
- The system correctly tracked only the largest (closest) face when multiple people were in frame.
- At fast walking speeds, the system could not keep up due to the ~200ms processing latency, causing the face to leave the frame.

2.7 DISCUSSION

The use of face detection on the ESP32-CAM showed that even a small device can handle complex tasks like security monitoring. When the room was well-lit and the person was at a medium distance, the module identified faces quickly and responded in real time. By linking the camera to a motor that pans or a function that captures images, we proved that the system can successfully connect what it sees to how it moves. This confirms that the built-in software on the ESP32-CAM is a practical choice for simple tasks that involve tracking human presence.

Nonetheless, the experiment also highlighted some clear limits in how much the processor can handle. In low light or in front of busy backgrounds, the system slowed down significantly, causing the video to become choppy. This happens because the small chip is working hard to capture images, compress files, and run detection math all at once. We also noticed that the camera sometimes mistook shadows or wall patterns for faces. These results show that the environment and the chip's limited power play a huge role in how accurately the system performs.

We also encountered some hardware issues regarding how the motor moved. Since the motor and the camera shared the same power source, the motor would sometimes jitter when it moved quickly. This was likely due to small drops in power whenever the camera worked extra hard. To make the system better in the future, we could give the motor its own battery, adjust the software settings, or add code to make the movements smoother. Overall, the project demonstrated the importance of balancing software needs with power and hardware limits when building smart devices.

2.8 RECOMMENDATIONS

To make the ESP32-CAM more reliable at finding faces, we need to do more than just change the clock speed. While slowing down the internal clock helps reduce signal noise and makes the camera more stable, it also makes the video feed much slower. A better approach would be to fine-tune other settings, such as the size of the image, the brightness, and the light sensitivity. By adjusting these factors together, we can find a good balance where the camera detects faces accurately without making the system lag.

We should also improve how the system makes decisions once it thinks it sees a face. Instead of having the motor move or the camera take a photo every single time the sensor is triggered, we can add specific rules to the code. For example, we could require the camera to see a face for several frames in a row before it actually reacts. Adding a short "cool-down" period between movements can also stop the system from jumping at shadows. These simple logic changes ensure the system only acts when it is certain, which prevents false alarms and keeps the mechanical parts from wearing out.

Finally, the entire setup would benefit from a more stable power supply and better timing for the motors. Since both the camera and the motor pull a lot of power at the same time, using a separate, dedicated 5V power source would help keep the system from stuttering or resetting. We could also use better mathematical formulas in our code to make the motor movements appear smoother rather than jerky. Following these steps will result in a

more professional device that processes information efficiently and responds reliably to human presence.

2.9 CONCLUSION

The completion of Task 2, shows the overall versatility of the ESP-32 CAM as a central hub for smart IoT applications. The integration of the cam's face detection capabilities demonstrates the modules potential for simple surveillance application by detecting human faces and additional actions from it such as servo movement and image capturing. By combining Wi-Fi based live video streaming and automated tracking, this module proves the seamless connection between microcontrollers, actuators and sensors. It establishes that a low-cost platform supported by precise parameter tuning serves as a robust foundation for scalable real-time operation.

3.0 REFERENCES

[1] N. Nadzelan, “Getting Started With ESP32-CAM,” *Cytron Technologies Malaysia*, Apr. 2, 2023. [Online]. Available: <https://my.cytron.io/tutorial/getting-started-with-esp32-cam>. Accessed: Apr. 29, 2026.

[2] Random Nerd Tutorials, “ESP32-CAM Video Streaming and Face Recognition with Arduino IDE,” Dec. 10, 2019. [Online]. Available: <https://randomnerdtutorials.com/esp32-cam-video-streaming-face-recognition-arduino-ide/>. Accessed: Apr. 29, 2026.

4.0 APPENDICES

5.0 ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Dr. Zulkifli Bin Zainal Abidin and Dr. Wahyu Sediono for their continuous support and for providing the essential resources and facilities required to carry out this project. We also extend our heartfelt appreciation to everyone who contributed to the success of this lab report through their valuable insights, assistance, and feedback.

6.0 STUDENTS DECLARATION

We hereby affirm that the work presented in this report is entirely our own. All sources used for background information and theory have been properly cited, and have strictly adhered to the principles of academic integrity throughout this report.